

PRACTICAL NO 08

PROGRAM: Write a C Program to implement Breadth First Search (BFS) in graph.

ALGORITHM:

1. Start with a graph represented by adjacency matrix.
2. Create a visited[] array and set all values = 0 (means not visited).
3. Create an empty queue.
4. Mark the start vertex as visited and put it in the queue.
5. While the queue is not empty:
 - a. Remove (dequeue) the first element from the queue → call it node.
 - b. Print node.
 - c. For every vertex i connected to node:
 - If i is not visited and adj[node][i] == 1 (edge exists), then mark i as visited and add it to the queue.
6. Repeat until queue is empty.

PROGRAM:

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int queue[MAX], front = 0, rear = -1;
```

```
int visited[MAX];
```

```
// BFS function
```

```
void bfs(int adj[MAX][MAX], int n, int start) {
```

```
    int i, node;
```

```
    // mark start node visited and put in queue
```

```

visited[start] = 1;
queue[++rear] = start;

printf("BFS Traversal: ");
while (front <= rear) {
    node = queue[front++]; // remove from queue
    printf("%d ", node);

    // visit all neighbours
    for (i = 0; i < n; i++) {
        if (adj[node][i] == 1 && visited[i] == 0) {
            queue[++rear] = i;
            visited[i] = 1;
        }
    }
}
printf("\n");
}

```

```

// Main
int main() {
    int n, adj[MAX][MAX], i, j, start;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }
}

```

```
}

printf("Enter starting vertex (0 to %d): ", n-1);
scanf("%d", &start);

// initialize visited array to 0
for (i = 0; i < n; i++) visited[i] = 0;

bfs(adj, n, start);

return 0;
}
```

SAMPLE INPUT :Enter number of vertices: 4

Enter adjacency matrix:

0 1 1 0

1 0 0 1

1 0 0 1

0 1 1 0

Enter starting vertex (0 to 3): 0

OUTPUT: BFS Traversal: 0 1 2 3